

IMPERIAL



MENG INDIVIDUAL PROJECT

IMPERIAL COLLEGE LONDON

DEPARTMENT OF COMPUTING

Implementation of Foundational Verification for Probabilistic Programs in Continuous Domain

Author:
Edwin Fernando

Supervisor:
Shing Hin Ho
Dr. Azalea Raad

Second Marker:
?

January 15, 2026

Abstract

Probabilistic Programming offers a principled way to decouple inference algorithms from the statistical models a programmer/statistician wishes to create. There are several languages (anglican hakaru, [?]) used predominantly by statisticians. But there are much wider applications in computer science, where arguments of correctness are probabilistic in nature. Examples include Zero Knowledge Proof Systems (Blockchain), Differential Privacy and Machine Learning. We can reason expressively and formally verify these arguments if the implementation was carried out in a probabilistic programming language/DSL, using Probabilistic Separation Logics (PSL). This project is meant as a proof of concept, aiming to be the first implementation of a PSL. Specifically in the Lean proof assistant, we implement the lilac separation logic using Iris-lean (ported from rocq) to provide ergonomic tactical support. We choose Lean for the significant body of measure-theoretic results present in Mathlib, required to establish soundness of the logic.

Given time I would also like to pursue formalisation of BaSL or a theory for symbolic execution using lilac or BaSL for probabilistic programs.

on top of the Iris framework, in Lean. offer an intuitive way to reason about probabilistic programs at a suitable level of abstraction.

Contents

1	Introduction	3
1.1	Objectives	3
1.2	Challenges	3
1.3	Contributions	3
2	Background	4
2.1	Lean	4
2.2	Probabilistic Program Semantics	4
2.3	Separation Logic	4
2.4	Measure Theory	4
2.4.1	Pi-lambda theorem	4
2.4.2	Disintegration theorem	4
2.5	Symbolic Execution tools	4
2.6	Iris and MoSeL	4
3	PROJECT X	5

Chapter 1

Introduction

Hello [?]

1.1 Objectives

1.2 Challenges

1.3 Contributions

Chapter 2

Background

2.1 Lean

A lot of background in Lean was required to undertake this project. The Lean development exercises the full expressivity of inductive types in Lean, for example and a thorough understanding of propositional vs definitional equality which is related to Proof irrelevance [Theorem proving for Lean4] and how that's implemented in Lean's type theory by special treatment of the lowest Universe [Mario's thesis]

Various representations of formal languages particularly the problem of how to represent variables in contexts, with strong guarantees from the types System. [Cite chlipala, HOAS, and lean-lang.org]

2.2 Probabilistic Program Semantics

This section is entirely a summary of the denotational semantics given to Bayesian Probabilistic programs given by Staton. The probabilistic programs we aim to provide verification tools for in this work do not go beyond the generality of the semantics presented by Staton. A brief overview of key ideas is provided below (but it is better to read the)

- A clear treatment of Staton's commutative semantics.
- The distinction and intuition behind the difference between s-finite and sigma-finite measures. Perhaps even why they're used.

2.3 Separation Logic

O'Hearn's original paper. PSL, Lilac, BaSL

It is unclear to me why an axiomatic semantics would need to be provided in terms of measurable maps from the Hilbert cube. I wonder whether measure theory is not abstract enough and perhaps synthetic measure theory may hold some intuitions/inspiration or simplifications

2.4 Measure Theory

2.4.1 Pi-lambda theorem

2.4.2 Disintegration theorem

2.5 Symbolic Execution tools

2.6 Iris and MoSeL

Chapter 3

PROJECT X

I want to note that I have not before seen the interpretation of a typed calculus, taking both the type and typing context as parameters to give the denotation of a term. This definition of terms with they're typing context, actually massively simplifies the lean development previous iterations, involved a type inference algorithm, which could fail and the denotation function was partial.

Possible goals of this project:

- Formalisation of Lilac with tactical support
- Lean interpreter for the language which is sound w.r.t to the denotational semantics
- The first case of Compositional Symbolic execution for probabilistic separation logic

CSE for the Bluebell separation logic with possible immediate application in industry for verifying correctness of Zero Knowledge Proof systems This could perhaps be a tactic, which actually invokes the symbolic execution to elaborate into a sequence of proof rules?

This is possibly not so hard

CSE for Lilac or BaSL. this is perhaps out of scope of project. But it connects to the first part of the project formalising Lilac/BaSL which is novel for being the first mechanisation of a separation logic for continuous probabilistic programs.

terms in context

Bibliography